

TASK 04 — Role-Based Authorization & API Permissions

Step 1 — Create Roles

Think of roles as permission groups.

Role	Example permissions
USER	View products
ADMIN	Create, update, delete products
MANAGER	Manager-specific operations

In a Spring Security application, these can be represented as:

```
public enum Role {  
    USER,  
    ADMIN,  
    MANAGER  
}
```

Step 2 — Assign a Default Role

When a new user registers, they should automatically receive:

New User → USER

Example:

Name: Ravi

Email: ravi@gmail.com

Password: ****

Role: USER

Important security rule

A normal user **must not be allowed to select ADMIN during registration.**

Bad approach:

```
{  
  "name": "Ravi",  
  "email": "ravi@gmail.com",  
  "password": "12345",  
  "role": "ADMIN"  
}
```

If your registration API accepts this, someone could register themselves as an administrator.

Better approach

The backend decides the role:

```
user.setRole(Role.USER);
```

So even if the client sends:

```
"role": "ADMIN"
```

the backend should **ignore it** for normal registration.

Step 3 — Create Role-Based APIs

USER

GET /api/v1/products

A user can **view products**.

For example:

GET /api/v1/products

Response:

```
[  
  {  
    "id": 1,  
    "name": "Laptop",  
    "price": 50000  
  },  
]
```

```
{  
  "id": 2,  
  "name": "Phone",  
  "price": 25000  
}  
]
```

ADMIN

An administrator gets additional permissions:

POST /api/v1/products

PUT /api/v1/products/{id}

DELETE /api/v1/products/{id}

1. Create product

POST /api/v1/products

Example:

```
{  
  "name": "Laptop",  
  "price": 50000  
}
```

Only an ADMIN should be allowed to perform this operation.

2. Update product

PUT /api/v1/products/1

This means:

Update the product whose ID is 1.

Example:

```
{  
  "name": "Dell Laptop",
```

```
"price": 55000
}
```

Again, only an authorized role should be allowed.

3. Delete product

```
DELETE /api/v1/products/1
```

This means:

Delete product ID 1.

This should be restricted to ADMIN.

Step 4 — Configure authorization

Spring Security Configuration

we can configure authorization using `@PreAuthorize`.

First enable it:

```
@Configuration
```

```
@EnableWebSecurity
```

```
@EnableMethodSecurity
```

```
public class SecurityConfig {
```

```
    @Bean
```

```
    public SecurityFilterChain securityFilterChain(
```

```
        HttpSecurity http) throws Exception {
```

```
        http
```

```
            .csrf(csrf -> csrf.disable())
```

```
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers(
```

```
                    "/api/v1/auth/register",
```

```
                    "/api/v1/auth/login"
```

```
                ).permitAll()
```

```

        .anyRequest().authenticated()

    );

    return http.build();
}
}

```

The important part is:

`@EnableMethodSecurity`

This allows you to use:

`@PreAuthorize(...)`

Step 5 — Test USER

USER can view products

`@GetMapping`

`@PreAuthorize("hasAnyRole('USER', 'ADMIN', 'MANAGER')")`

```

public List<Product> getProducts() {
    return productRepository.findAll();
}

```

Now:

USER → 

ADMIN → 

MANAGER → 

can access:

GET /api/v1/products

USER cannot create products

Create API:

`@PostMapping`

`@PreAuthorize("hasRole('ADMIN')")`

```
public Product createProduct(  
    @RequestBody Product product) {  
    return productRepository.save(product);  
}
```

Now:

USER → ❌

ADMIN → ✅

MANAGER → ❌

A USER attempting:

POST /api/v1/products

will receive: 403 Forbidden

because the user is authenticated but doesn't have the required role.

USER cannot delete products

```
@DeleteMapping("/{id}")  
@PreAuthorize("hasRole('ADMIN')")  
public String deleteProduct(  
    @PathVariable Long id) {  
    productRepository.deleteById(id);  
    return "Product deleted successfully";  
}
```

Permission:

USER → ❌

ADMIN → ✅

MANAGER → ❌

USER cannot manage users

For example, suppose you create:

GET /api/v1/users

You can protect it:

```
@GetMapping
```

```
@PreAuthorize("hasRole('ADMIN')")
```

```
public List<User> getUsers() {  
    return userRepository.findAll();  
}
```

Therefore:

USER → ❌

ADMIN → ✅

MANAGER → ❌

Step 6 — Test ADMIN

ADMIN — Create

```
@PostMapping
```

```
@PreAuthorize("hasRole('ADMIN')")
```

```
public Product createProduct(  
    @RequestBody Product product) {  
    return productRepository.save(product);  
}
```

Request: POST /api/v1/products

Body:

```
{  
    "name": "Laptop",  
    "price": 50000
```

```
}
```

ADMIN: Allowed

USER: 403 Forbidden

ADMIN — Read

```
@GetMapping
```

```
@PreAuthorize("hasAnyRole('USER', 'ADMIN', 'MANAGER')")
```

```
public List<Product> getProducts() {  
    return productRepository.findAll();  
}
```

ADMIN: Allowed

ADMIN — Update

```
@PutMapping("/{id}")
```

```
@PreAuthorize("hasRole('ADMIN')")
```

```
public Product updateProduct(  
    @PathVariable Long id,  
    @RequestBody Product product) {  
    Product existing =  
        productRepository.findById(id)  
            .orElseThrow(  
                () -> new RuntimeException(  
                    "Product not found"  
                )  
            );  
    existing.setName(product.getName());  
    existing.setPrice(product.getPrice());  
    return productRepository.save(existing);  
}
```

```
}
```

Request: PUT /api/v1/products/1

Body:

```
{
```

```
  "name": "Dell Laptop",
```

```
  "price": 60000
```

```
}
```

ADMIN: Allowed

USER: Forbidden

ADMIN — Delete

```
@DeleteMapping("/{id}")
```

```
@PreAuthorize("hasRole('ADMIN')")
```

```
public String deleteProduct(
```

```
    @PathVariable Long id) {
```

```
    productRepository.deleteById(id);
```

```
    return "Product deleted successfully";
```

```
}
```

Request: DELETE /api/v1/products/1

ADMIN: Allowed

USER: Forbidden

Step 7 — Test MANAGER

This is just one possible permission model; your application's actual business requirements can define different manager permissions.

Manager API Example

Create:

```

@GetMapping("/reports")
@PreAuthorize("hasRole('MANAGER')")
public String getReports() {
    return "Manager reports";
}

```

Only: MANAGER

can access: GET /api/v1/products/reports

Complete Permission Table

Your application can now look like this:

Operation	USER	ADMIN	MANAGER
View products	✓	✓	✓
Create product	✗	✓	✗
Update product	✗	✓	✗
Delete product	✗	✓	✗
Manage users	✗	✓	✗
View reports	✗	✗	✓

Complete Controller

Putting the above together:

```

@RestController
@RequestMapping("/api/v1/products")
public class ProductController {
    private final ProductRepository productRepository;

    public ProductController(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
}

```

```

// USER + ADMIN + MANAGER

@GetMapping
@PreAuthorize("hasAnyRole('USER', 'ADMIN', 'MANAGER')")
public List<Product> getProducts() {
    return productRepository.findAll();
}

// ADMIN ONLY

@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public Product createProduct(
    @RequestBody Product product) {
    return productRepository.save(product);
}

// ADMIN ONLY

@PutMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public Product updateProduct(
    @PathVariable Long id,
    @RequestBody Product product) {
    Product existing =
        productRepository.findById(id)
            .orElseThrow(
                () -> new RuntimeException(
                    "Product not found"
                )
            );
};

```

```

        existing.setName(product.getName());
        existing.setPrice(product.getPrice());
        return productRepository.save(existing);
    }

    // ADMIN ONLY
    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public String deleteProduct(
        @PathVariable Long id) {
        productRepository.deleteById(id);
        return "Product deleted successfully";
    }

    // MANAGER ONLY
    @GetMapping("/reports")
    @PreAuthorize("hasRole('MANAGER')")
    public String reports() {
        return "Manager reports";
    }
}

```

Step 8 — Test Unauthorized Access

This means a normal USER is already logged in, but tries to access an API that requires ADMIN.

Example

Suppose we have:

Ravi

Role = USER

And this API is restricted to administrators:

```
@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public String deleteProduct(@PathVariable Long id) {
    productRepository.deleteById(id);
    return "Product deleted";
}
```

Ravi sends:

DELETE /api/v1/products/10

Authorization: Bearer USER_JWT

Spring Security checks:

JWT

↓

User authenticated?

↓

YES

↓

Role = USER

↓

Required role = ADMIN

↓

USER does not have ADMIN permission

↓

403 Forbidden

Why 403:

Because Ravi **is authenticated**, but he **doesn't have permission** to perform the operation.

Example in Postman

Login as USER

POST <http://localhost:8083/api/v1/auth/login>

Body:

```
{  
  "email": "ravi@gmail.com",  
  "password": "Ravi@123"  
}
```

You receive a JWT: eyJhbGciOiJIUzI1NiJ9....

Try ADMIN API

Copy the JWT into Postman:

Authorization



Bearer Token



USER_JWT

Then: DELETE <http://localhost:8083/api/v1/products/1>

Your controller has: `@PreAuthorize("hasRole('ADMIN')")`

Response:

HTTP 403 Forbidden

That is the expected result.

Step 9 — Authentication vs Authorization

401 — Not Authenticated

401 Unauthorized means the request does not have valid authentication credentials.

For example:

Client



GET /api/v1/products



No JWT



Spring Security



Who is this user?



Cannot authenticate



401 Unauthorized

Example: GET /api/v1/products

with **no Authorization header**.

Response: 401 Unauthorized

403 — Authenticated but Not Authorized

Now suppose the user has a valid JWT.

Client



JWT



Spring Security



JWT is valid



User = Ravi

Role = USER



API requires ADMIN



USER doesn't have permission



403 Forbidden

So:

USER + valid JWT + ADMIN API



403 Forbidden

Simple Real-Life Example

Think of an office building.

401

You enter the building without showing your ID.

Security guard says:

"I don't know who you are."

That's similar to:

401 → Not authenticated

403

You show your employee ID.

The guard knows:

"You are Ravi."

But you try to enter the **Admin Room**, and your ID doesn't give you access.

That's:

403 → Authenticated but not authorized

Spring Boot Example

Public API

```
@GetMapping("/public")  
  
public String publicApi() {  
    return "Public API";  
}
```

Anyone can access it.

Authenticated API

```
@GetMapping("/profile")  
  
@PreAuthorize("isAuthenticated()")  
  
public String profile() {  
    return "User profile";  
}
```

A valid authenticated user is required.

ADMIN API

```
@DeleteMapping("/users/{id}")  
  
@PreAuthorize("hasRole('ADMIN')")  
  
public String deleteUser(@PathVariable Long id) {  
  
    return "User deleted";  
}
```

Only:

ROLE_ADMIN

can access it.

Step 10 — Permission Matrix

View Products

GET /api/v1/products

All three roles can view products:

USER 

MANAGER 

ADMIN 

Spring Security:

@GetMapping

```
@PreAuthorize("hasAnyRole('USER', 'MANAGER', 'ADMIN')")
```

```
public List<Product> getProducts() {  
    return productRepository.findAll();  
}
```

A normal user can read/view product information.

Create Product

POST /api/v1/products

According to your matrix:

USER 

MANAGER 

ADMIN 

Code:

@PostMapping

```
@PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")
```

```
public Product createProduct(@RequestBody Product product) {  
    return productRepository.save(product);  
}
```

So:

USER → 403 Forbidden

MANAGER → Allowed

ADMIN → Allowed

Update Product

PUT /api/v1/products/{id}

Your matrix says:

USER 

MANAGER 

ADMIN 

Code:

```
@PutMapping("/{id}")
```

```
@PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")
```

```
public Product updateProduct(
```

```
    @PathVariable Long id,
```

```
    @RequestBody Product product) {
```

```
    Product existingProduct =
```

```
        productRepository.findById(id)
```

```
        .orElseThrow(
```

```
            () -> new RuntimeException(
```

```
                "Product not found"
```

```
            )
```

```
        );
```

```
    existingProduct.setName(product.getName());
```

```
    existingProduct.setPrice(product.getPrice());
```

```
    return productRepository.save(existingProduct);
```

```
}
```

Result

USER → 403 Forbidden

MANAGER → Allowed

ADMIN → Allowed

Delete Product

DELETE /api/v1/products/{id}

Your matrix says:

USER 

MANAGER 

ADMIN 

Therefore, **only ADMIN can delete products.**

```
@DeleteMapping("/{id}")
```

```
@PreAuthorize("hasRole('ADMIN')")
```

```
public String deleteProduct(@PathVariable Long id) {
```

```
    productRepository.deleteById(id);
```

```
    return "Product deleted successfully";
```

```
}
```

Result:

USER → 403

MANAGER → 403

ADMIN → Allowed

Manage Users

This is the most sensitive operation in your matrix.

USER 

MANAGER 

ADMIN 

For example:

GET /api/v1/users

can be restricted to ADMIN:

@GetMapping

@PreAuthorize("hasRole('ADMIN')")

```
public List<User> getUsers() {  
    return userRepository.findAll();  
}
```

Only an administrator can access it.

Complete API Mapping

Based on your matrix, your controller authorization should be:

View

@PreAuthorize("hasAnyRole('USER', 'MANAGER', 'ADMIN')")

Create

@PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")

Update

@PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")

Delete

@PreAuthorize("hasRole('ADMIN')")

Manage Users

@PreAuthorize("hasRole('ADMIN')")